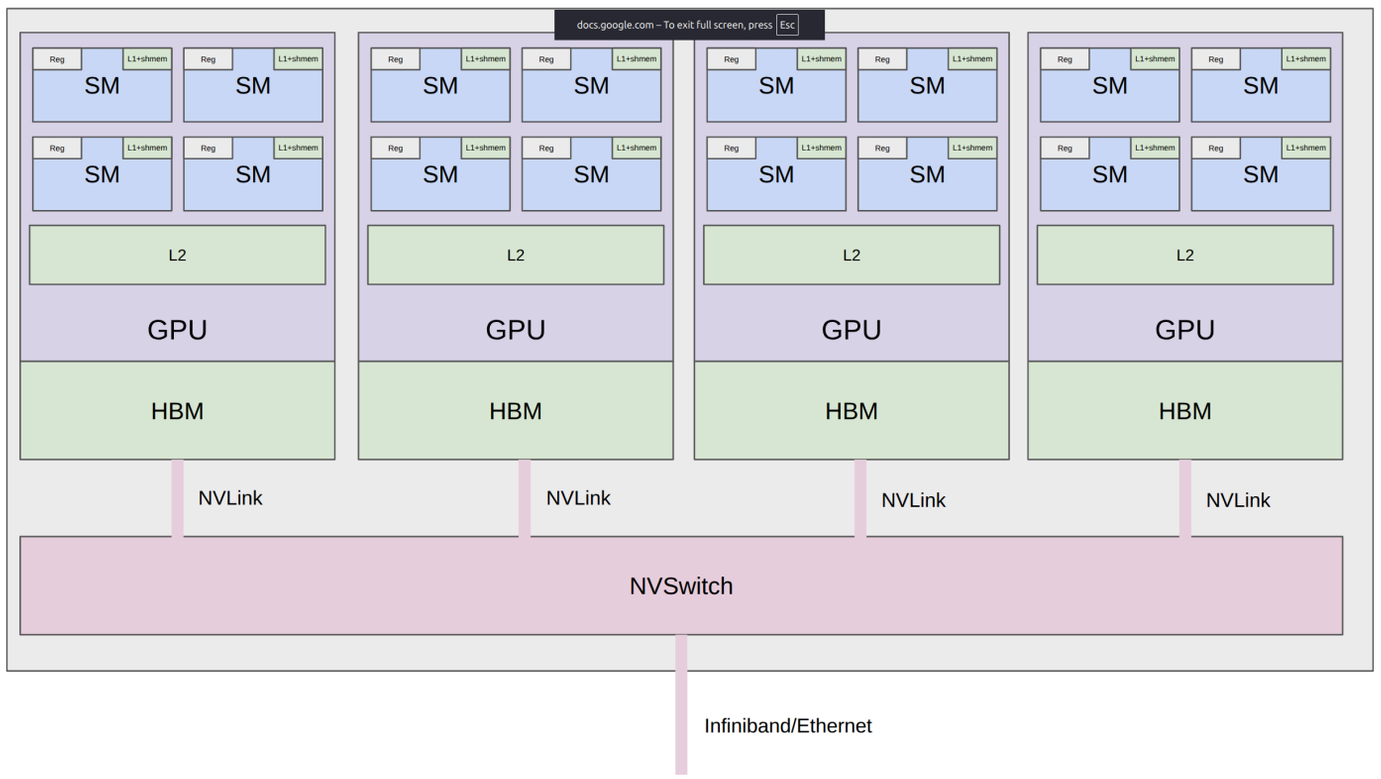
Lecture 7: parallelism

Last week: parallelism within a single GPU
This week: parallelism across multiple GPUs



> 图：GPU 节点结构概览：两块 GPU 各含多个 SM（每个 SM 配 Reg 寄存器堆与 L1+shmem 一级缓存/共享内存）以及共享的 L2 缓存，并连接各自的 HBM 显存；两卡经交换芯片（switch）互联，再与 CPU（含其 DRAM 内存）相连。

> 第 7 讲：并行 上周：单 GPU 内部的并行 本周：跨多个 GPU 的并行

In both cases, **compute** (arithmetic logic units) is far from inputs/outputs (**data**).
Unifying theme: orchestrate computation to avoid data transfer bottlenecks

> 这两种情况下，计算单元（算术逻辑单元）都远离输入/输出（数据）。 统一主题：编排计算以规避数据搬运瓶颈

Generalized hierarchy:

- Single node, single GPU: L1 cache / shared memory (fastest)
- Single node, single GPU: HBM
- Single node, multi-GPU: NVLink/NVSwitch
- Multi-node, multi-GPU: Infiniband/Ethernet (slowest)

> 广义的层级结构： 单节点、单 GPU：L1 缓存 / 共享内存（最快） 单节点、单 GPU：HBM 单节点、多 GPU：NVLink/NVSwitch 多节点、多 GPU：Infiniband/以太网（最慢）

Last week: reduce memory accesses via fusion/tiling
This week: reduce communication across GPUs/nodes via replication/sharding

> 上周：通过融合/分块（tiling）减少内存访问 本周：通过复制/分片减少跨 GPU/节点的通信

Why do multi-GPU?
1. Your parameters (optimizer state + gradients + activations) don't fit on a single GPU.
2. You want to use more GPUs (more FLOPs) to train faster.

> 为什么要用多 GPU？ 你的参数（优化器状态 + 梯度 + 激活值）放不进单张 GPU。 你想用更多 GPU（更多 FLOPs）来加快训练。

[stdout for this lecture]

> [本讲的 stdout]

Part 1: building blocks of distributed communication/computation

Part 2: distributed training

Walk through bare-bones implementations of each strategy on deep MLPs.
Recall that MLPs are the compute bottleneck in Transformers, so this is representative.

> 第一部分：分布式通信/计算的基本构件 第二部分：分布式训练 在深度 MLP 上逐个走读每种策略的最简实现。 回顾：MLP 是 Transformer 中的计算瓶颈，因此这具有代表性。

What's missing?

- Communication/computation overlap
- More general models (with attention, etc.)
- Other forms of parallelism (e.g., sequence parallelism, expert parallelism, combinations)
- Jax/TPUs: just define the model, the sharding strategy, and the Jax compiler handles the rest [levanter]
- But we're doing PyTorch so you can see how one builds up from the primitives

> 缺了什么？ 通信/计算重叠 更一般的模型（含注意力等） 其他形式的并行（如序列并行、专家并行及其组合） Jax/TPU：只需定义模型和分片策略，其余交给 Jax 编译器 [levanter] 但我们用 PyTorch，这样你能看到如何从这些原语一步步搭建起来

Summary

- Many ways to parallelize: data (batch), tensor/expert (width), pipeline (depth), sequence (length)
- Data parallelism: DDP (all-reduce), FSDP/ZeRO (all-gather + reduce-scatter)
- Tensor parallelism: requires very fast interconnects (e.g., NVLink)
- Pipeline parallelism: can work with slow interconnects, but need to work to reduce pipeline bubbles
- Can **re-compute** or store in **memory** or store in another GPUs memory and **communicate**
- Hardware is getting faster, but will always want bigger models, so will have this hierarchical structure

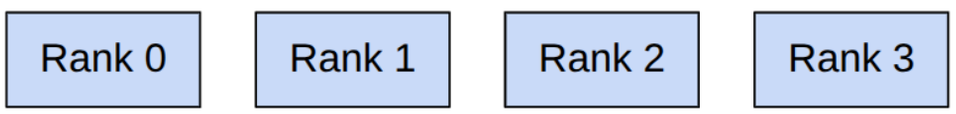
> 总结 并行方式有多种：数据并行（批次维度）、张量/专家并行（宽度）、流水线并行（深度）、序列并行（长度） 数据并行：DDP（all-reduce）、FSDP/ZeRO（all-gather + reduce-scatter） 张量并行：需要极快的互连（如 NVLink） 流水线并行：可以在慢速互连下工作，但需要设法减少流水线气泡 可以重新计算、存放在内存中，或存到另一块 GPU 的内存中再通信 硬件越来越快，但我们总想要更大的模型，所以这种层级结构会一直存在

Collective operations are the conceptual primitives used for distributed programming [article]

- These are classic in the parallel programming literature from the 1980s.
- *Collective* means that you specify a general communication pattern across many devices.
- This can be better/faster than managing point-to-point communication yourself.

> 集合通信操作是分布式编程所使用的概念原语 [文章] 它们在 20 世纪 80 年代的并行编程文献中已是经典。 集合意味着你指定的是跨多台设备的通用通信模式。 这可能比自己管理点对点通信更好/更快。

Setup:



> 图：四个并排的方框，从左到右标注 Rank 0、Rank 1、Rank 2、Rank 3，代表分布式训练中的四个进程/设备编号。

- **Rank**: a particular device/GPU (e.g., 0, 1, 2, 3)
- **World size**: total number of devices (e.g., 4)

> 设定： Rank：某个具体的设备/GPU（如 0、1、2、3） World size：设备总数（如 4）

Operations:

- Broadcast, scatter, gather, reduce (foundations)
- All-gather, reduce-scatter, all-reduce (workhorse)
- All-to-all (for MoEs)

> 操作： Broadcast、scatter、gather、reduce（基础） All-gather、reduce-scatter、all-reduce（主力） All-to-all（用于 MoE）

Broadcast: copy from rank 0 to all ranks

> Broadcast：从 rank 0 拷贝到所有 rank

Minor use case: rank 0 loads initial checkpoint and broadcasts to all ranks

> 次要用例：rank 0 加载初始 checkpoint 并广播给所有 rank

Scatter tensor on rank 0 to all ranks

> 把 rank 0 上的张量 scatter 到所有 rank

Note: stepping stone to understanding reduce-scatter

> 注：这是理解 reduce-scatter 的垫脚石

Gather pieces from all ranks to rank 0 (opposite of scatter)

> 把所有 rank 的分片 gather 到 rank 0（scatter 的逆操作）

Note: stepping stone to understanding all-gather

> 注：这是理解 all-gather 的垫脚石

Reduce pieces from all ranks to rank 0, applying some operation (e.g., sum, min, max)

> 对来自所有 rank 的分片施加某种运算（如 sum、min、max）后 reduce 到 rank 0

Note: stepping stone to understanding all-reduce

> 注：这是理解 all-reduce 的垫脚石

All-gather: perform gather to all ranks, not just rank 0

> All-gather：gather 到所有 rank，而不只是 rank 0

Use case: each rank holds parameter shard, gather to get full parameters for forward pass

> 用例：每个 rank 持有参数分片，gather 得到完整参数用于前向传播

Reduce-scatter: perform reduce on each dimension, scatter results

> Reduce-scatter：在每个维度上执行 reduce，再 scatter 结果

Use case: after backward pass, sum gradients from different data shards, but distribute storage

> 用例：反向传播后，对不同数据分片的梯度求和，但分散存储

All-reduce = reduce-scatter + all-gather
Use case: after backward pass, sum gradients from different data shards, but replicate full parameters
Breaking all-reduce into reduce-scatter + all-gather allows for flexibility (e.g., ZeRO/FSDP)

> 用例：反向传播后，对不同数据分片的梯度求和，并复制完整参数 把 all-reduce 拆成 reduce-scatter + all-gather 可以带来灵活性（如 ZeRO/FSDP）

All-to-all: each rank sends each other rank some tensor (most general)

> All-to-all：每个 rank 向其他每个 rank 发送某个张量（最通用的形式）

Notes:

- Useful for MoEs: each rank has split of data and subset of experts; need to route data to experts
- For balanced splits, all-to-all looks like transpose
- Also handles unbalanced splits (but want splits to be as balanced as possible)

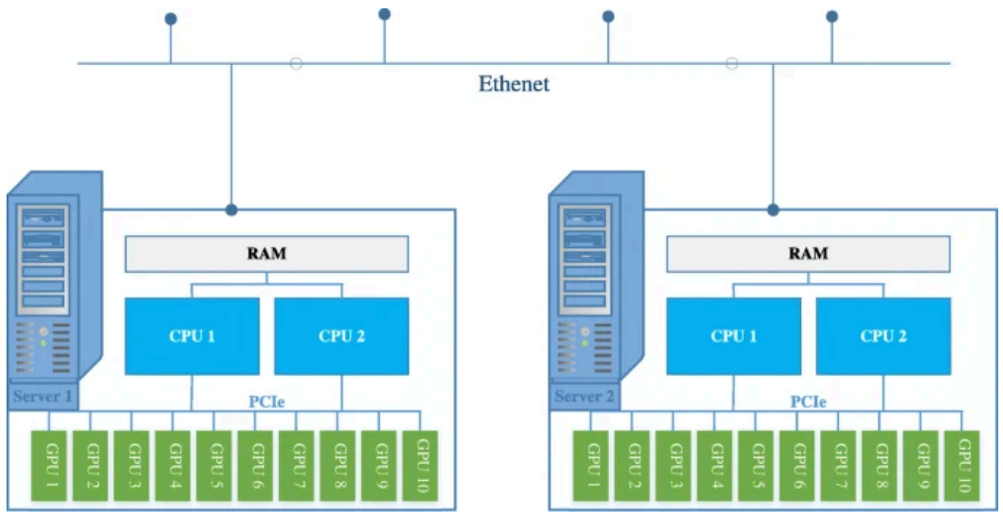
> 注： 对 MoE 很有用：每个 rank 持有数据的一个切分和一部分专家；需要把数据路由到专家 对均衡的切分，all-to-all 看起来就像转置 也能处理不均衡的切分（但应尽量让切分均衡）

Way to remember the terminology:

- Reduce: performs some associative/commutative operation (sum, min, max)
- Scatter is inverse of gather
- All: means destination is all devices

> 记忆这套术语的方法： Reduce：执行某种可结合/可交换的运算（sum、min、max） Scatter 是 gather 的逆操作 All：意味着目标是所有设备

Classic (in the home):

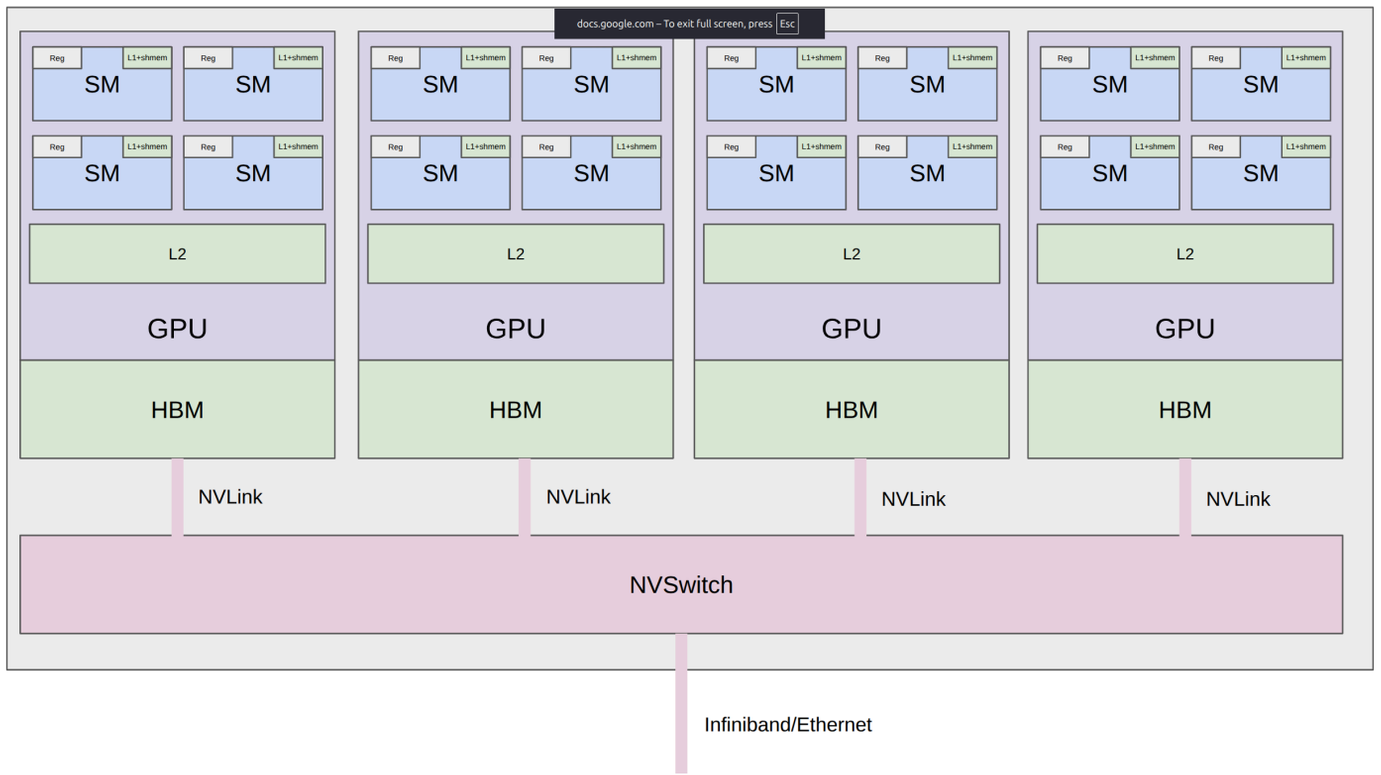


> 图：论文图 1——多个子图对比 “GPU (hidden)” 与 “GPU (non-hidden)” 两种配置下吞吐量随训练轮数的变化，横轴为 epoch 数（Number of epochs），纵轴为吞吐量（Throughput）。

- GPUs on same node communicate via a PCI(e) bus (v7.0, 16 lanes => 242 GB/s) [article]
- GPUs on different nodes communicate via Ethernet (~200 MB/s)

> 经典场景（家庭环境）： 同一节点内的 GPU 通过 PCI(e) 总线通信（v7.0，16 条通道 => 242 GB/s） [文章] 不同节点的 GPU 通过以太网通信（约 200 MB/s）

Modern (in the data center):



> 图：GPU 节点示意（Google 幻灯片截图）： 每块 GPU 内含寄存器（Reg）、SRAM、SMs 与 HBM；GPU 之间由 NVLink 交换机互连，经 NIC 接入网络，并经 PCIe 连到 host（M 个 CPU 核与内存）。

> 现代场景（数据中心）：

Typical setup:

- 8 GPUs per node, connected by NVLink to an NVSwitch (B200s' NVLink 5.0 gets 1.8 TB/s; HBM was 8 TB/s)
- 256 nodes per pod, connected by Infiniband (via PCIe -> HCA / Infiniband NIC -> Infiniband cable) (~0.05 TB/s)
- N pods per cluster / datacenter, connected by Ethernet (via PCIe -> CPU)

> 典型配置： 每节点 8 块 GPU，经 NVLink 连到 NVSwitch（B200 的 NVLink 5.0 达 1.8 TB/s；HBM 为 8 TB/s） 每个 pod 256 个节点，经 Infiniband 连接（路径为 PCIe -> HCA / Infiniband 网卡 -> Infiniband 线缆）（约 0.05 TB/s） 每个集群/数据中心 N 个 pod，经以太网连接（路径为 PCIe -> CPU）

Bypassing the CPU:

- Ethernet requires passing through the CPU (copying data to kernel socket buffer, build TCP packets, copy to NIC ring buffer)
- Remote Direct Memory Access (RDMA): allows one GPU to directly read/write another GPU's memory without involving the CPU

- Infiniband supports RDMA, but standard Ethernet does not

> 绕过 CPU： 以太网需要经过 CPU（把数据拷贝到内核 socket 缓冲区、构建 TCP 包、拷贝到网卡环形缓冲区） 远程直接内存访问（RDMA）： 允许一块 GPU 直接读/写另一块 GPU 的内存，无需 CPU 介入 Infiniband 支持 RDMA，而标准以太网不支持

Advancements:

- GB200/GB300 NVL72: 8 GPUs per tray, 9 trays per rack -> 72 GPUs in one NVLink domain
- RDMA over Converged Ethernet (RoCE): Ethernet bypasses CPU, similar but cheaper/weaker than Infiniband, used by Meta

> 新进展： GB200/GB300 NVL72：每个托盘 8 块 GPU，每个机架 9 个托盘 -> 一个 NVLink 域内共 72 块 GPU RDMA over Converged Ethernet（RoCE）： 以太网绕过 CPU，与 Infiniband 类似但更便宜/性能更弱，Meta 在用

NVIDIA Collective Communication Library (NCCL)

NCCL translates collective operations into low-level packets that are sent between GPUs. [talk]

- Detects topology of hardware (e.g., number of nodes, switches, NVLink/PCIe)
- Optimizes the path between GPUs
- Launches GPU kernels to send/receive data

> NVIDIA 集合通信库（NCCL） NCCL 把集合通信操作翻译成在 GPU 之间发送的底层数据包。 [演讲] 探测硬件拓扑（如节点数、交换机、NVLink/PCIe） 优化 GPU 之间的传输路径 启动 GPU kernel 来收发数据

PyTorch distributed library (`torch.distributed`) [documentation]

- Provides clean interface for collective operations (e.g., `all_gather_into_tensor`)
- Supports multiple backends for different hardware: gloo (CPU), nccl (GPU)
- Also supports higher-level algorithms (e.g., `FullyShardedDataParallel`) [not used in this course]

> PyTorch 分布式库（torch.distributed） [文档] 为集合通信操作提供简洁的接口（如 all_gather_into_tensor） 支持多种后端以适配不同硬件：gloo（CPU）、nccl（GPU） 也支持更高层的算法（如 FullyShardedDataParallel）[本课程未使用]

Let's walk through some examples.

> 我们来看几个例子。

Indeed, all-reduce = reduce-scatter + all-gather!

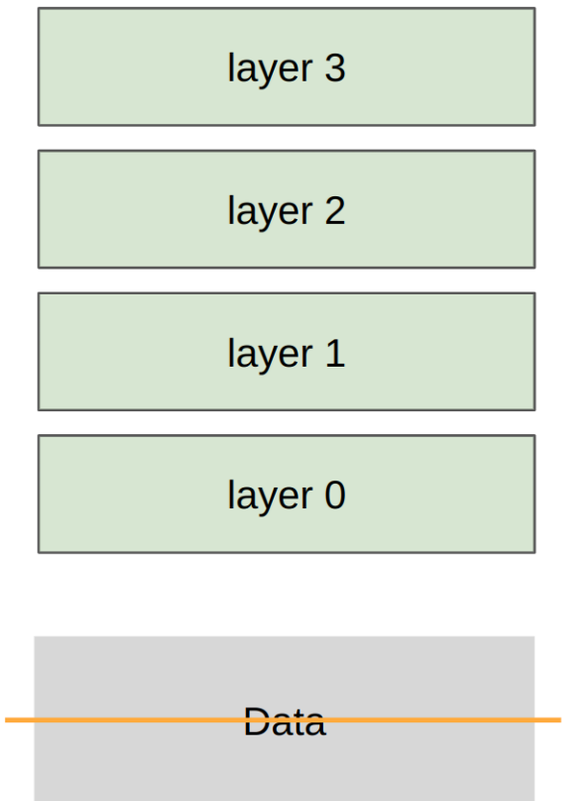
> 没错，all-reduce = reduce-scatter + all-gather！

How fast does communication happen?

> 通信到底有多快?

References:
[How to reason about collective operations]
[Sample benchmarking code]

> 参考文献： [如何分析集合通信操作] [基准测试示例代码]



> 图：数据并行示意图：同一模型（layer 0 至 layer 3 各层）复制到 4 个设备上各持一份副本，输入数据被切分（Data）后分给各设备并行处理。

Sharding strategy: each rank gets a slice of the data

> 分片策略：每个 rank 拿到一部分数据

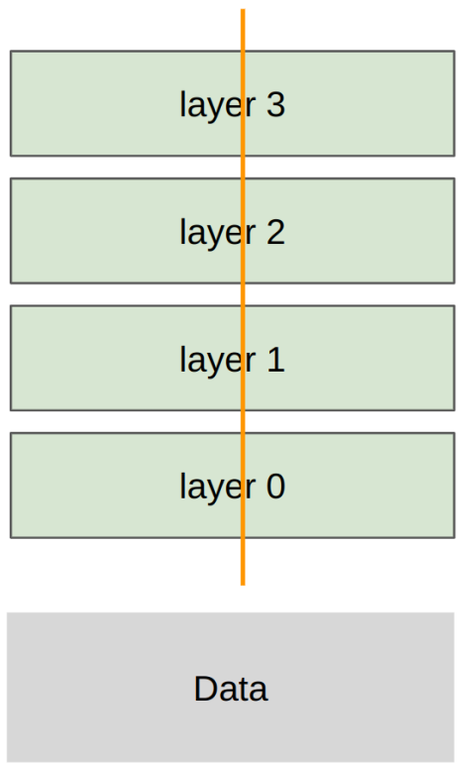
Notes:

- Losses are different across ranks (computed on local data)
- Gradients are all-reduced to be the same across ranks
- Therefore, parameters remain the same across ranks

> 注： 各 rank 的损失不同（在本地数据上计算） 梯度经 all-reduce 后在各 rank 上一致 因此参数在各 rank 上保持一致

Next time: FSDP/ZeRO: use all-gather and reduce-scatter to avoid holding all parameters in memory

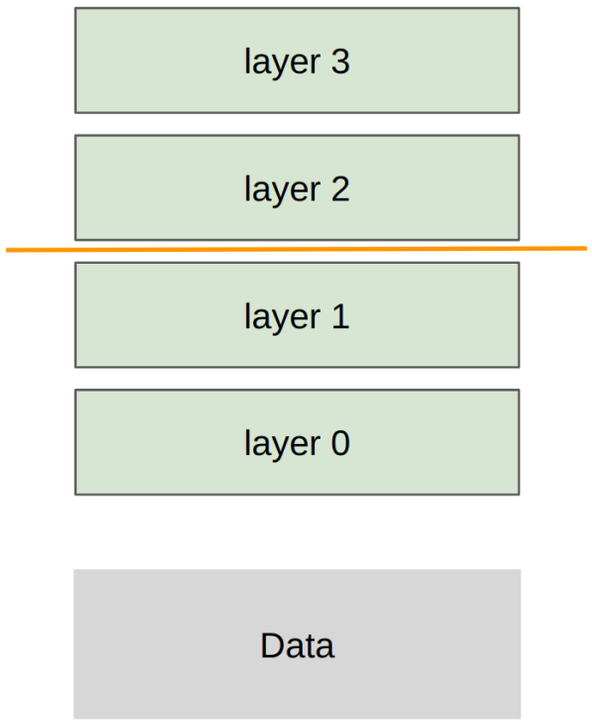
> 下次讲：FSDP/ZeRO：用 all-gather 和 reduce-scatter 避免把全部参数都留在内存中



> 图：张量并行示意图：从上到下依次堆叠 layer 3、layer 2、layer 1、layer 0 四层，层堆栈下方是数据（Data）；一条竖直剖分线贯穿全部四层，表示把每一层内部沿同一维度切开、分布到多个设备上。

Sharding strategy: each rank gets part of each layer, transfer all data/activations

> 分片策略：每个 rank 拿到每一层的一部分，传输全部数据/激活值



> 图：流水线并行的微批次执行时间线——GPU 0 上依次执行各微批次的前向 F1、F2，GPU 1 上执行对应反向 B2、B1；F 表示前向传播、B 表示反向传播，前后向交错排布形成流水线。

Sharding strategy: each rank gets subset of layers, transfer all data/activations

> 分片策略：每个 rank 拿到一部分层，传输全部数据/激活值

Not handled: overlapping communication/computation to eliminate pipeline bubbles

> 未涉及：通过通信/计算重叠来消除流水线气泡